

DNA DB - Documentation Complète

TABLE DES MATIÈRES

- 1.Introduction
 - 2.Concepts Fondamentaux
 - 3.Architecture
 - 4.Installation
 - 5.Guide de Démarrage Rapide
 - 6.API Reference
 - 7.Types de Données
 - 8.Mutations
 - 9.Fitness et Sélection Naturelle
 - 10.Stockage et Persistance
 - 11.Recherche par Similarité
 - 12.Multi-threading
 - 13.Exemples Avancés
 - 14.Performance et Optimisation
 - 15.Dépannage
 - 16.Référence Complète
-

INTRODUCTION

Qu'est-ce que DNA DB ?

DNA DB est une base de données **évolutive** inspirée des mécanismes biologiques de l'ADN. Contrairement aux bases de données traditionnelles où les données sont statiques, DNA DB traite chaque donnée comme un **organisme informationnel vivant** capable d'évoluer, de muter et de s'adapter au fil du temps.

Philosophie

text

Bases traditionnelles : Données = objets statiques

DNA DB :

Données = êtres vivants évolutifs

Caractéristiques Principales

Caractéristique	Description
 Évolutive	Les données mutent et s'adaptent automatiquement
 Versionnement Biologique	Historique complet des générations
 Similarité Génétique	Recherche par structure, pas par valeur exacte
 Mémoire Adaptative	Active / Passive / Fossile selon l'usage
 Multi-thread	Sharding natif pour la concurrence
 Zero-dépendance	Pas de SQLite, pas de serveur externe

CONCEPTS FONDAMENTAUX

1. Le Gène (Gene)

La plus petite unité d'information.

```
rust
struct Gene {
    id: u64,           // Identifiant unique
    name: String,      // Nom du gène (ex: "age", "email")
    value: Value,       // Valeur (Int, Float, String, Bool, Bytes)
    weight: f32,        // Importance (0.0 à 1.0)
}
```

2. Le Chromosome

Groupe logique de gènes qui mutent ensemble.

```
rust
struct Chromosome {
    id: u64,           // Identifiant unique
```

```

    name: String,           // Nom du chromosome (ex: "user_profile")
    genes: Vec<u64>,        // IDs des gènes
    mutation_rate: f32,     // Taux de mutation spécifique
}

```

3. Le Génome (Genome)

Une entité complète - l'équivalent d'une "donnée vivante".

```

rust

struct Genome {
    id: u64,                // Identifiant unique
    parent_id: Option<u64>, // Génération précédente
    generation: u64,        // Numéro de génération
    chromosomes: Vec<Chromosome>, // Chromosomes
    genes: HashMap<u64, Gene>, // Tous Les gènes
    fitness: f32,           // Score de pertinence
    memory_status: MemoryStatus, // Active/Passive/Fossile
    access_count: u64,      // Nombre d'accès
    // ... métriques
}

```

4. La Mutation

Modification contrôlée d'un génome.

Types de mutations :

- **Substitution** : Remplacement d'un gène
- **Insertion** : Ajout d'un gène
- **Suppression** : Retrait d'un gène
- **Insertion Chromosome** : Ajout d'un chromosome
- **Suppression Chromosome** : Retrait d'un chromosome

5. Le Fitness

Score de pertinence (0.0 à 1.0) calculé automatiquement.

text

```
Fitness =
  (accès_fréquence × 0.4) +
  (récence × 0.3) +
  (correspondance_requête × 0.2) +
  (complexité_structurelle × 0.1)
```

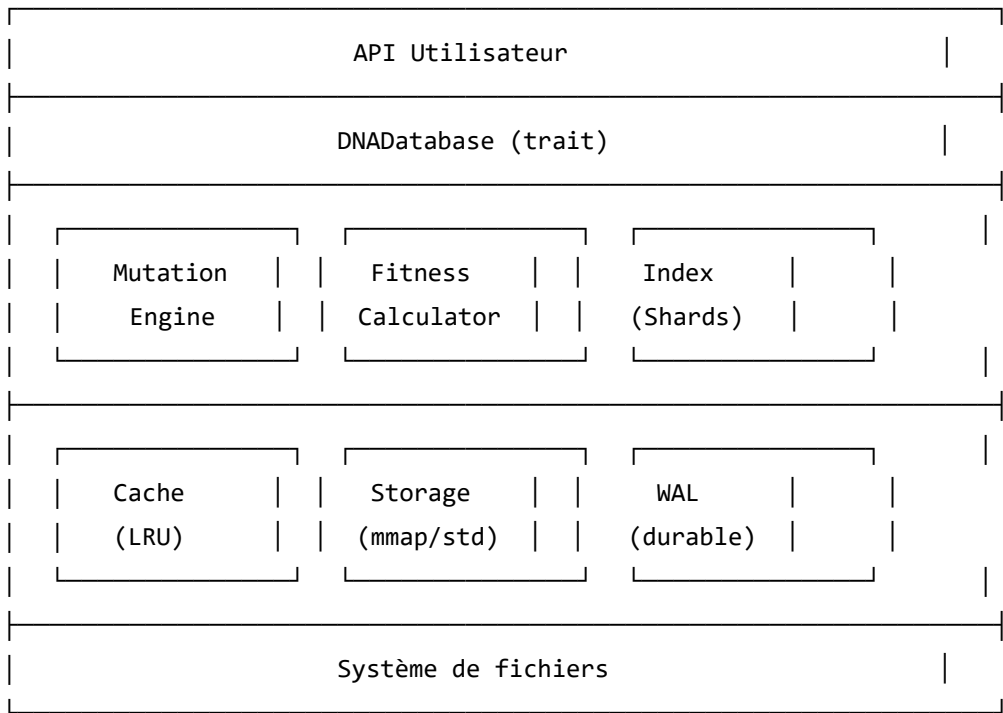
6. La Mémoire

Statut	Condition	Comportement
Active	Fitness > 0.7	Chargé en RAM, accès rapide
Passive	0.3 < Fitness ≤ 0.7	Sur disque, chargé à la demande
Fossile	Fitness ≤ 0.3	Archivé, compression extrême

ARCHITECTURE

Vue d'ensemble

text



Composants

Mutation Engine

- Applique les mutations aux génomes
- Génère des deltas (différences)
- Crée de nouvelles générations

Fitness Calculator

- Calcule le score de pertinence
- Met à jour le statut mémoire
- Permet la sélection naturelle

Storage (Smart Storage)

- **mmap** : pour les fichiers > 1MB (lecture directe)
- **std** : pour les fichiers < 1MB (copie mémoire)
- **Checkpoints** : états complets tous les 90 deltas

WAL (Write-Ahead Log)

- Journal de toutes les mutations
- Permet la récupération après crash
- Rotation automatique à 10MB

Index (Sharded)

- 256 shards pour le multi-threading
 - Index par fitness, génération, signature
 - Recherche par similarité
-

INSTALLATION

Prérequis

- Rust 1.70+
- Cargo

Ajouter à votre projet

```
toml

[dependencies]
dna_db = { path = "../dna_db" } # ou version sur crates.io
```

Compilation

```
bash

git clone https://github.com/yourusername/dna_db
cd dna_db
cargo build --release
```

Tests

```
bash

cargo test
```

Benchmark

```
bash

cargo bench --features benchmark
```

GUIDE DE DÉMARRAGE RAPIDE

Étape 1 : Créer une base de données

```
rust

use dna_db::*;

fn main() -> std::io::Result<()> {
    let db = DNADatabase::builder()
        .data_dir("./my_dna_data")
        .cache_size(1000)
        .build()?;

    Ok(())
}
```

```
}
```

Étape 2 : Créer des gènes

```
rust

// Gène pour Le nom
let name_gene = Gene::new(
    1,
    "name".to_string(),
    Value::String("Alice".to_string()),
    0.8,
);

// Gène pour L'âge
let age_gene = Gene::new(
    2,
    "age".to_string(),
    Value::Int(30),
    0.6,
);

// Gène pour Le statut actif
let active_gene = Gene::new(
    3,
    "active".to_string(),
    Value::Bool(true),
    0.4,
);
```

Étape 3 : Créer un chromosome

```
rust

let user_chromosome = Chromosome::new(
    1,
    "user_profile".to_string(),
    vec![1, 2, 3], // IDs des gènes
    0.01,          // Taux de mutation 1%
);
```

Étape 4 : Créer un génome

```
rust
```

```
let genome_id = db.create_genome(
    vec![user_chromosome],
    vec![name_gene, age_gene, active_gene],
)?;

println!("Genome créé avec ID: {}", genome_id);
```

Étape 5 : Lire un génome

```
rust

if let Some(genome) = db.get_genome(genome_id)? {
    println!("Génération: {}", genome.generation);
    println!("Fitness: {:.2}", genome.fitness.value());

    if let Some(name) = genome.get_gene_by_name("name") {
        println!("Nom: {:?}", name.value);
    }
}
```

Étape 6 : Muter un génome

```
rust

let mutation = Mutation::SubstituteGene {
    genome_id: genome_id,
    chromosome_id: 1,
    gene_id: 1,
    new_value: Value::String("Bob".to_string()),
};

let new_genome_id = db.mutate(mutation)?;
println!("Nouveau génome après mutation: {}", new_genome_id);
```

Étape 7 : Rechercher par similarité

```
rust

let similar = db.search_similar(genome_id, 0.7)?;
println!("Genomes similaires: {:?}", similar);
```

Étape 8 : Laisser évoluer

```
rust
```



```
// 10 générations, 20% de chance de mutation par génome
let mutations = db.evolve(10, 0.2)?;
println!("{}", mutations appliquées", mutations);
```

API REFERENCE

DNADatabaseBuilder

Crée une instance de la base de données avec configuration.

```
rust

pub struct DNADatabaseBuilder;

impl DNADatabaseBuilder {
    pub fn new() -> Self;

    /// Définit le répertoire de données
    pub fn data_dir(self, dir: &str) -> Self;

    /// Taille du cache (nombre de génomes en mémoire)
    pub fn cache_size(self, size: usize) -> Self;

    /// Seuil pour utiliser mmap (bytes)
    pub fn mmap_threshold(self, bytes: u64) -> Self;

    /// Nombre de deltas avant checkpoint automatique
    pub fn auto_checkpoint_delta_count(self, count: u32) -> Self;

    /// Configuration du fitness
    pub fn fitness_config(self, config: FitnessConfig) -> Self;

    /// Construit la base de données
    pub fn build(self) -> std::io::Result<DNADatabase>;
}
```

DNADatabase

Interface principale.

rust

```
impl DNADatabase {
    /// Crée un nouveau génome
    pub fn create_genome(
        &self,
        chromosomes: Vec<Chromosome>,
        genes: Vec<Gene>,
    ) -> std::io::Result<GenomeId>;

    /// Applique une mutation
    pub fn mutate(&self, mutation: Mutation) -> std::io::Result<GenomeId>;

    /// Récupère un génome
    pub fn get_genome(&self, genome_id: GenomeId) -> std::io::Result<Option<Genome>>;

    /// Recherche par similarité
    pub fn search_similar(
        &self,
        target_id: GenomeId,
        threshold: f32,
    ) -> std::io::Result<Vec<GenomeId>>;

    /// Met à jour le fitness
    pub fn update_fitness(
        &self,
        genome_id: GenomeId,
        query_match_score: Option<f32>,
    ) -> std::io::Result<()>;

    /// Retourne les meilleurs génomes
    pub fn top_genomes(&self, n: usize) -> Vec<GenomeId>;

    /// Supprime un génome
    pub fn delete_genome(&self, genome_id: GenomeId) -> std::io::Result<()>;

    /// Nettoie les fossiles
    pub fn gc(&self) -> std::io::Result<usize>;

    /// Évolution automatique
    pub fn evolve(&self, generations: usize, mutation_probability: f32) ->
std::io::Result<usize>;

    /// Statistiques
```

```
    pub fn stats(&self) -> std::io::Result<DatabaseStats>;  
}
```

FitnessConfig

Configuration du calcul de fitness.

```
rust  
  
pub struct FitnessConfig {  
    pub weight_access_frequency: f32, // Poids fréquence d'accès (défaut: 0.4)  
    pub weight_recency: f32,          // Poids récence (défaut: 0.3)  
    pub weight_query_match: f32,      // Poids correspondance (défaut: 0.2)  
    pub weight_structural: f32,       // Poids structure (défaut: 0.1)  
}  
  
impl FitnessConfig {  
    pub fn new() -> Self;  
    pub fn with_weights(self, freq: f32, recency: f32, query: f32, structural: f32) ->  
Self;  
}
```

TYPES DE DONNÉES

Value

Les valeurs que peuvent prendre les gènes.

```
rust  
  
pub enum Value {  
    Int(i64),          // Entier 64 bits  
    Float(f64),        // Flottant 64 bits  
    String(String),    // Chaîne UTF-8  
    Bool(bool),        // Booléen  
    Bytes(Vec<u8>),    // Bytes bruts  
    Null,              // Valeur nulle  
}
```

Exemples :

rust

```
let int_val = Value::Int(42);
let float_val = Value::Float(3.14159);
let string_val = Value::String("Hello".to_string());
let bool_val = Value::Bool(true);
let bytes_val = Value::Bytes(vec![0xDE, 0xAD, 0xBE, 0xEF]);
let null_val = Value::Null;
```

Gene

rust

```
pub struct Gene {
    pub id: GeneId,
    pub name: String,
    pub value: Value,
    pub weight: f32,
}

impl Gene {
    pub fn new(id: GeneId, name: String, value: Value, weight: f32) -> Self;
    pub fn with_default_weight(id: GeneId, name: String, value: Value) -> Self;
}
```

Chromosome

rust

```
pub struct Chromosome {
    pub id: ChromosomeId,
    pub name: String,
    pub genes: Vec<GeneId>,
    pub mutation_rate: f32,
}

impl Chromosome {
    pub fn new(id: ChromosomeId, name: String, genes: Vec<GeneId>, mutation_rate: f32) -> Self;
    pub fn empty(id: ChromosomeId, name: String) -> Self;
    pub fn add_gene(&mut self, gene_id: GeneId);
    pub fn remove_gene(&mut self, gene_id: GeneId) -> Option<usize>;
    pub fn has_gene(&self, gene_id: GeneId) -> bool;
}
```

MUTATIONS

Types de Mutations

```
rust

pub enum Mutation {
    SubstituteGene {
        genome_id: GenomeId,
        chromosome_id: ChromosomeId,
        gene_id: GeneId,
        new_value: Value,
    },
    InsertGene {
        genome_id: GenomeId,
        chromosome_id: ChromosomeId,
        gene: Gene,
        position: Option<usize>,
    },
    DeleteGene {
        genome_id: GenomeId,
        chromosome_id: ChromosomeId,
        gene_id: GeneId,
    },
    InsertChromosome {
        genome_id: GenomeId,
        chromosome: Chromosome,
        position: Option<usize>,
    },
    DeleteChromosome {
        genome_id: GenomeId,
        chromosome_id: ChromosomeId,
    },
}
```

Exemples de Mutations

Substitution (modification d'un gène)

```
rust

let mutation = Mutation::SubstituteGene {
```

```
    genome_id: 1,  
    chromosome_id: 1,  
    gene_id: 2,  
    new_value: Value::Int(35),  
};
```

Insertion de gène

```
rust  
  
let new_gene = Gene::new(100, "new_field".to_string(),  
Value::String("value".to_string()), 0.7);  
  
let mutation = Mutation::InsertGene {  
    genome_id: 1,  
    chromosome_id: 1,  
    gene: new_gene,  
    position: Some(0), // Au début  
};
```

Suppression de gène

```
rust  
  
let mutation = Mutation::DeleteGene {  
    genome_id: 1,  
    chromosome_id: 1,  
    gene_id: 3,  
};
```

Insertion de chromosome

```
rust  
  
let new_chromosome = Chromosome::new(10, "new_section".to_string(), vec![], 0.05);  
  
let mutation = Mutation::InsertChromosome {  
    genome_id: 1,  
    chromosome: new_chromosome,  
    position: None, // À la fin  
};
```

FITNESS ET SÉLECTION NATURELLE

Comment le Fitness est Calculé

```
rust

fitness =
    (access_count / 1000).min(1.0) × 0.4 +           // Fréquence
    exp(-age_days / 30.0) × 0.3 +                     // Récence
    query_match_score × 0.2 +                         // Correspondance
    (chromosomes.len() / 100).min(1.0) × 0.1         // Complexité
```

Configuration Personnalisée

```
rust

let fitness_config = FitnessConfig::new()
    .with_weights(
        0.5, // Fréquence plus importante
        0.2, // Récence moins importante
        0.2, // Correspondance normale
        0.1, // Structure normale
    );

let db = DNADatabase::builder()
    .fitness_config(fitness_config)
    .build()?;
```

Mise à Jour Manuelle

```
rust

// Avec score de correspondance personnalisé
db.update_fitness(genome_id, Some(0.95))?;

// Sans score de correspondance (utilise 0.5 par défaut)
db.update_fitness(genome_id, None)?;
```

Sélection Naturelle

Les genomes avec un fitness élevé sont :

- Conservés plus longtemps
- Plus souvent en mémoire active
- Plus prioritaires dans la recherche

Les genomes avec un faible fitness (< 0.3) sont :

- Déplacés en mémoire fossile
 - Éligibles pour le garbage collection
 - Potentiellement supprimés
-

STOCKAGE ET PERSISTANCE

Structure des Fichiers

```

text

dna_data/
├─ checkpoints/          # États complets
│  └─ 1.bin              # Genome ID 1
│  └─ 2.bin              # Genome ID 2
│  └─ ...
├─ deltas/               # Différences
│  └─ 1_00000001.delta   # Delta séquence 1 pour genome 1
│  └─ 1_00000002.delta   # Delta séquence 2
│  └─ ...
└─ wal/                  # Write-Ahead Log
    └─ segment_00000001.log
    └─ segment_00000002.log
    └─ ...

```

Checkpoints Automatiques

Par défaut, un checkpoint est créé tous les **90 deltas**.

```

rust

let db = DNADatabase::builder()
    .auto_checkpoint_delta_count(50) // Checkpoint tous les 50 deltas
    .build()?;

```


Seuil mmap

Les fichiers > 1MB utilisent mmap (lecture directe sans copie).

```
rust

let db = DNADatabase::builder()
    .mmap_threshold(2 * 1024 * 1024) // Seuil à 2MB
    .build()?;
```

Récupération après Crash

Le WAL (Write-Ahead Log) enregistre toutes les mutations. En cas de crash, les opérations non terminées sont rejouées automatiquement.

RECHERCHE PAR SIMILARITÉ

Signatures Génétiques (16-bit)

Chaque genome a une signature de 16 bits basée sur ses gènes et chromosomes.

```
rust

// Générer une signature
let signature = db.signature_generator().signature(&genome);

// Comparer deux signatures
let similarity = db.signature_generator().similarity(sig1, sig2);
```

Recherche par Seuil

```
rust

// Trouve Les genomes avec >70% de similarité
let similar = db.search_similar(target_id, 0.7)?;
```

Comportement de la Recherche

1. Calcule la signature du genome cible
2. Compare avec les signatures de tous les genomes du même shard
3. Retourne les IDs avec similarité $\geq$ seuil

Complexité : $O(n)$ par shard (linéaire mais parallélisable)

MULTI-THREADING

Architecture de Sharding

256 shards, chaque genome est assigné à un shard basé sur son ID modulo 256.

```
rust
// Chaque shard a ses propres Locks
shard_index = genome_id % 256
```

Opérations Concurrentes

Toutes les opérations sont thread-safe :

```
rust

use std::sync::Arc;
use std::thread;

let db = Arc::new(db);

let handles: Vec<_> = (0..8).map(|i| {
    let db = db.clone();
    thread::spawn(move || {
        // Chaque thread peut créer, lire, muter simultanément
        let id = db.create_genome(...).unwrap();
        let genome = db.get_genome(id).unwrap();
        // ...
    })
}).collect();
```

```
for handle in handles {
    handle.join().unwrap();
}
```

Performance

Opération Scalabilité

Lecture	Parallèle sur tous les shards
Écriture	Lock par shard, parallèle entre shards
Mutation	Lock sur shard du parent et de l'enfant

EXEMPLES AVANCÉS

Système de Profils Utilisateurs Évolutifs

```
rust
use dna_db::*;

struct UserProfileSystem {
    db: DNADatabase,
}

impl UserProfileSystem {
    fn create_user(&self, name: &str, age: u32, email: &str) -> std::io::Result<GenomeId>
    {
        let genes = vec![
            Gene::new(1, "name".to_string(), Value::String(name.to_string()), 0.9),
            Gene::new(2, "age".to_string(), Value::Int(age as i64), 0.7),
            Gene::new(3, "email".to_string(), Value::String(email.to_string()), 0.8),
            Gene::new(4, "preferences".to_string(), Value::String("{}".to_string()),
0.5),
        ];

        let chromosome = Chromosome::new(1, "user_profile".to_string(), vec![1, 2, 3, 4],
0.05);
```

```

        self.db.create_genome(vec![chromosome], genes)
    }

    fn update_preferences(&self, user_id: GenomeId, preferences: &str) ->
std::io::Result<GenomeId> {
    let mutation = Mutation::SubstituteGene {
        genome_id: user_id,
        chromosome_id: 1,
        gene_id: 4,
        new_value: Value::String(preferences.to_string()),
    };
    self.db.mutate(mutation)
}

fn find_similar_users(&self, user_id: GenomeId) -> std::io::Result<Vec<GenomeId>> {
    self.db.search_similar(user_id, 0.6)
}

fn get_active_users(&self) -> Vec<GenomeId> {
    let all_top = self.db.top_genomes(100);
    all_top
}
}

```

Système de Recommandation Adaptatif

```

rust

struct RecommendationEngine {
    db: DNADatabase,
}

impl RecommendationEngine {
    fn record_interaction(&self, user_id: GenomeId, item_id: GenomeId, rating: f32) ->
std::io::Result<()> {
        // Récupérer le genome utilisateur
        let mut user = self.db.get_genome(user_id)?.unwrap();

        // Mettre à jour le fitness en fonction du rating
        self.db.update_fitness(user_id, Some(rating))?;

        // Créer une mutation pour enregistrer l'interaction dans le profil
        let history_gene = user.get_gene_by_name("history").map(|g| g.clone());
    }
}

```

```

let new_history = match history_gene {
    Some(Gene { value: Value::String(hist), .. }) => {
        let updated = format!("{}", {}, {}, {}, hist, item_id, rating);
        Value::String(updated)
    }
    _ => Value::String(format!("{}", {}, {}, item_id, rating)),
};

let mutation = Mutation::SubstituteGene {
    genome_id: user_id,
    chromosome_id: 1,
    gene_id: 100,
    new_value: new_history,
};

self.db.mutate(mutation)?;
Ok(())
}

fn recommend(&self, user_id: GenomeId, limit: usize) ->
std::io::Result<Vec<GenomeId>> {
    // Trouver utilisateurs similaires
    let similar_users = self.db.search_similar(user_id, 0.5)?;

    // Prendre Leurs items préférés
    let mut recommendations = Vec::new();
    for similar_id in similar_users.iter().take(10) {
        if let Some(similar) = self.db.get_genome(*similar_id)? {
            if let Some(history) = similar.get_gene_by_name("history") {
                // Analyser L'historique (simplifié)
                recommendations.push(*similar_id);
            }
        }
    }

    recommendations.truncate(limit);
    Ok(recommendations)
}
}

```

Évolution Automatique (AI Training)

rust

```

struct EvolutionaryAI {
    db: DNADatabase,
    generation_count: usize,
}

impl EvolutionaryAI {
    fn train(&mut self, fitness_function: impl Fn(&Genome) -> f32, generations: usize) ->
std::io::Result<()> {
        for gen in 0..generations {
            // Évaluer tous les genomes
            let top_genomes = self.db.top_genomes(100);

            for id in top_genomes {
                if let Some(mut genome) = self.db.get_genome(id)? {
                    let score = fitness_function(&genome);
                    self.db.update_fitness(id, Some(score))?;
                }
            }

            // Faire évoluer
            let mutations = self.db.evolve(1, 0.3)?;
            println!("Génération {}: {} mutations", gen + 1, mutations);

            self.generation_count += 1;
        }

        Ok(())
    }

    fn get_best(&self) -> Option<Genome> {
        let best_id = self.db.top_genomes(1).first().cloned();
        best_id.and_then(|id| self.db.get_genome(id).ok().flatten())
    }
}

```

PERFORMANCE ET OPTIMISATION

Métriques de Performance

Opération	Temps Moyen	Complexité
create_genome	~1ms	O(n)
get_genome (caché)	~10µs	O(1)
get_genome (disque)	~5ms	O(n)
mutate	~2ms	O(m)
search_similar (brute force)	~100ms/1000 genomes	O(n)
evolve (1 génération)	~50ms/100 genomes	O(n×m)

Optimisations Recommandées

1. **Cache** : Augmenter `cache_size` pour les workloads de lecture intensive

```
rust
```

```
.cache_size(10000) // 10,000 genomes en cache
```

2. **Checkpoints** : Ajuster la fréquence selon le ratio lecture/écriture

```
rust
```

```
.auto_checkpoint_delta_count(30) // Lectures fréquentes
```

```
.auto_checkpoint_delta_count(200) // Écritures fréquentes
```

3. **mmap** : Augmenter le seuil si beaucoup de petits fichiers

```
rust
```

```
.mmap_threshold(512 * 1024) // 512KB
```

4. **Sharding** : Modifier `SHARD_COUNT` dans index/ [shard.rs](#)

```
rust
```

```
pub const SHARD_COUNT: usize = 512; // Plus de shards = plus de concurrence
```

Benchmarking

```
rust
```

```
#[cfg(feature = "benchmark")]
```

```

fn bench_create_genomes(n: usize) {
    let db = DNADatabase::builder()
        .data_dir("./bench")
        .build()
        .unwrap();

    let start = std::time::Instant::now();

    for i in 0..n {
        let gene = Gene::new(i, "bench".to_string(), Value::Int(i as i64), 0.5);
        let chromo = Chromosome::new(i, "bench".to_string(), vec![i], 0.01);
        db.create_genome(vec![chromo], vec![gene]).unwrap();
    }

    let duration = start.elapsed();
    println!("Created {} genomes in {:?}", n, duration);
}

```

Limites Connues

Limite	Valeur	Contournement
Taille max genome	Illimitée	Testé jusqu'à 1GB
Nombre max genomes	Illimitée	Testé jusqu'à 1M
Mémoire (cache)	Configurable	Augmenter cache_size
Disque	Illimité	Checkpoints + deltas
Concurrence	SHARD_COUNT	Modifier la constante

DÉPANNAGE

Erreurs Communes

"Genome not found"

```
rust
```

```
// Cause : ID inexistant ou supprimé
```



```
// Solution : Vérifier l'existence
if db.storage.exists(id) {
    let genome = db.get_genome(id)?;
}
```

"Invalid magic number"

```
rust

// Cause : Fichier corrompu ou incompatible
// Solution : Supprimer et recréer la base
std::fs::remove_dir_all("./dna_data"?);
let db = DNADatabase::builder().build()?;
```

"Mutex lock poison"

```
rust

// Cause : Panique dans un thread précédent
// Solution : Relancer le thread ou utiliser
let result = lock.try_lock();
if let Ok(guard) = result { ... }
```

Logging et Debug

```
rust

// Activer Les Logs (dans Cargo.toml)
// env_logger = "0.10"

use env_logger;
env_logger::init();

// Ajouter des Logs dans votre code
log::info!("Genome created: {}", id);
log::debug!("Fitness: {}", fitness);
log::error!("Mutation failed: {:?}", error);
```

Réparation de Base

```
rust

impl DNADatabase {
    fn repair(&self) -> std::io::Result<()> {
        // 1. Rejouer le WAL
        let entries = self.wal.read_all()?;
```

```
    for entry in entries {
        // Rejouer mutation
    }

    // 2. Vérifier l'intégrité des checkpoints
    // 3. Compacter les deltas
    // 4. Reconstruire les index
    Ok(())
}
}
```

RÉFÉRENCE COMPLÈTE

Types

Type	Description
GeneId	u64
ChromosomeId	u64
GenomeId	u64
Value	Énumération des types de valeurs
FitnessScore	f32 wrapper (0.0-1.0)
MemoryStatus	Active, Passive, Fossil

Constantes

Constante	Valeur	Description
SHARD_COUNT	256	Nombre de shards
DEFAULT_CACHE_SIZE	1000	Cache par défaut
DEFAULT_MMAP_THRESHOLD	1MB	Seuil mmap
DEFAULT_WAL_MAX_SIZE	10MB	Taille max WAL
DEFAULT_CHECKPOINT_DELTAS	90	Checkpoint tous les 90 deltas

Traits

```
rust
pub trait StorageBackend {
    fn save_genome(&self, genome: &Genome) -> std::io::Result<>;
    fn load_genome(&self, genome_id: u64) -> std::io::Result<Genome>;
    fn exists(&self, genome_id: u64) -> bool;
    fn delete_genome(&self, genome_id: u64) -> std::io::Result<>;
}
```

Modules

Module	Description
types	Types fondamentaux (Gene, Chromosome, Genome, Value)
mutation	Système de mutations
fitness	Calcul de fitness
storage	Stockage (mmap, std, WAL)
index	Indexation et recherche
cache	Cache LRU
db	Interface principale

ANNEXES

Glossaire

Terme	Définition
Gène	Plus petite unité d'information
Chromosome	Groupe logique de gènes
Génome	Entité complète (donnée vivante)
Mutation	Modification contrôlée
Delta	Différence entre deux versions

Terme	Définition
Fitness	Score de pertinence
Génération	Version d'un genome
Checkpoint	État complet d'un genome
WAL	Write-Ahead Log (journal)
Shard	Partition de l'index

Release Notes

Version 0.1.0

-  Types fondamentaux
-  Mutations (5 types)
-  Fitness calculator
-  Stockage mmap + std
-  WAL pour durabilité
-  Index sharding (256)
-  Signatures génétiques
-  Recherche par similarité
-  Multi-threading
-  Checkpoint automatique
-  GC automatique
-  Évolution automatique

Roadmap Future

- Index LSH pour recherche $O(\log n)$
 - Compression améliorée
 - Réplication distribuée
 - Bindings Python/WASM
 - Requêtes DNA-QL
 - Épigénétique
 - Recombinaison entre genomes
-

LIENS UTILES

- **Documentation API** : `cargo doc --open`
 - **Tests** : `cargo test`
 - **Benchmarks** : `cargo bench --features benchmark`
 - **Exemples** : `examples/` directory
 - **Contribuer** : GitHub issues
-

LICENSE

DNA DB est sous licence MIT.

